

Improve CC Search Accessibility

Project Synopsis

The CC Search website is a tool that is available to everyone on the internet, but its accessibility is quite low. The reasons for its low accessibility are varied in nature which affects different sections of people. Due to this reduced accessibility, it is not possible for everyone to completely utilize all the features that CC search provides and thus it becomes imperative to fix this issue. The final deliverable of this project will be a highly accessible website which everyone on the web can access from any device or region and which fully accommodates every visitor to enable them to utilize the search to its fullest.

Project Plan

The Problem

The CC search as a tool is very powerful and is highly useful, but it lacks accessibility. To highlight the problem let's look at a few of the issues with the present search:

1. The website is only available in English. This makes it very difficult for people from other regions to make full use of the features provided by the website.
2. A lot of segments of the website take a considerably high time to load (even on fast connections) due to the page fetching a lot of resources that might not necessarily be used at the moment. This makes it very difficult for users who don't have a high-speed internet connection to satisfactorily use the website.
3. The website was not presently designed with keeping mobile devices, particularly in mind. This makes some of the features stick out like a sore thumb when using in such a mode, again leaving out a considerable amount of users in an area of discomfort. Also, the website cannot be used in a keyboard-only mode and requires a lot of interactions with the pointer.
4. The website lacks any audio feature, any special support for people requiring screen readers and such to account for the differently-abled populace, this too, in turn, results in reduced accessibility.

These points encompass in essence the major problems associated with the accessibility of the present search which will be addressed in this project.

The Solution

This project will mainly address the above mentioned major problems. The ways in which they will be handled are elegant and descriptively detailed below:

1. Integrate translations with cc search (vue-i18n and transifex with the cc chooser):

What exactly is i18n and how is it different from l10n?

I18n is often misrepresented as Localization (L10n) and sometimes even Translation. I18n is product development focused so that in the case of software, one code base is made capable of supporting worldwide languages and locale-specific formatting and behaviors. It's a critical and sometimes underestimated step that is needed for successful and scalable L10n. Conversely, Localization makes a product specific to a target market or region(s), including translation of the interface and possible adaptation of terminologies and more.

CC search is used by a very large number of people with considerably different backgrounds which also include catering to people from completely different regions and communicating in different languages. The present cc search is available only in English, which limits accessibility to the section of people who don't necessarily communicate in English. Therefore to cater to the needs of those people as well we will have to internationalize the cc search website.

Following are some of the benefits of internationalization:

- a. Increase reach: There is a considerable populace for whom communicating in English is still not viable. But this does not mean that these people should not get the benefits of cc search. So internationalizing the website will greatly improve the reach of CC search.
- b. Improve Credibility: Due to the internationalization of the website the visitors will have a far more pleasant experience than without one, this, in turn, increases the trust factor among the users and give a huge boost to the credibility of the site
- c. Increased Engagements: Making the website more accessible to such people through internationalization will increase visitor participation. This, in turn, will lead to better feedback and response from the users allowing us to give future users a richer experience and much better facilities.
- d. SEO benefits: The online competition landscape is more competitive than it's ever been and won't be slowing up anytime soon! The need for SEO relevancy is higher than ever to continue driving traffic to the website. Also, SEO keywords are the most competitive in English, but in languages other than English, there is significantly less competition. This is natural as we've seen there is less content created in other languages than English. Therefore, if the site is multilingual, we can achieve a much better user reach through SEO optimizations.

The CC search website is written with Vue.js as the frontend framework, therefore the support which comes along with the use of Vue-i18n just simply overwhelms the other 2 options in its favor. The vue-i18n plugin comes with its own official Vue certified official documentation and provides a great synergy between the vue framework and itself.

To completely internationalize the website we will have to achieve the following goals:

- Developing software to be independent of a specific language or limiting character set, and independent from cultural conventions (e.g., date display, time display)
- Locale Frameworks
- Achieving Unicode compliance
- Elimination of hard-coded text (strings) in the code
- Removal and reworking of concatenated strings
- Support for the collation of data
- Support for bi-directional languages, such as Hebrew and Arabic

Where does transifex come into the picture?

Transifex is the software platform responsible for crowdsourcing translations for our translations repository as it is not pragmatic to manually translate every string into the multiple different languages we will be catering to. So to make this work easier we will be using transifex to crowdsource our translations which in turn will enable continuous internationalization without having to involve the manual pain of translating each string ourselves.

Integrating translations:

Vue provides a bunch of options to maintain our translations:

- inline as data in javascript code
- js files
- JSON files
- `<script>` section in a Vue file

While each of these options has its own merits and demerits, for a project the size of CC search, it much better to use the JSON format than the other 3. The reason why inline and js options are discarded is that it is not a good practice to combine the business logic with frontend framework making inline a really bad idea and also that the management of translations in a JS file is much tedious and will it will also be a pain to troubleshoot later.

The reason why JSON is preferred over `<script>` section in a Vue.js file is that the advantages it gives are manifold. It is far easier to maintain JSON files than to keep track of the vue files. Also, the translations that we will be receiving from transifex will most likely be in JSON format and so we can directly use it with minimal further formatting or structuring (if any at all) whereas when we are maintaining them as a JS file or in vue file it will require a great deal of data formatting, the complexity of which is not worth the advantage gained.

Some more advantages of using JSON are:

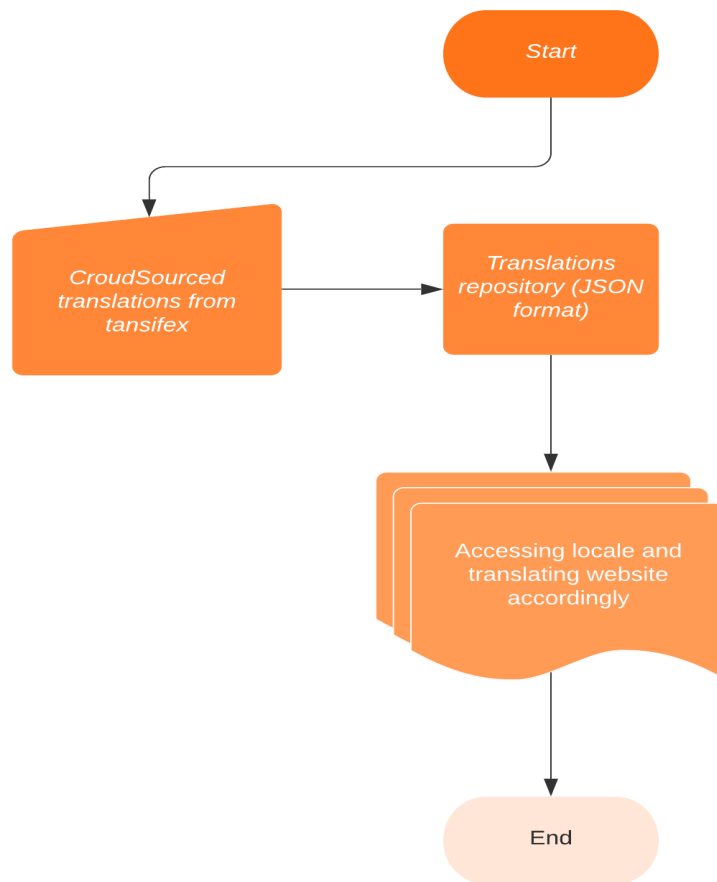
- Similar to the js files but more portable. We can still import/require them directly in your code or load them from a server.
- Validating JSON files are easy and there are tools that can help manage the translations. With an editor like BabelEdit it's impossible to create invalid files.
- You can keep one or many languages in one file — but I'd really recommend using one file per language. It's easier to reference other translations. This increases the consistency of the app.

We can use the above method for internationalization and for maintaining the translations.

So presently the proposed flow of integrating translations will be:

- a. Analyze the website and separate the different components whose strings will have to be translated.
- b. Attach a string identifier with each string. The string identifier will be made based on the descriptive nature or usage of the string.
- c. Make a template translations file which will have the English translations and also serve as the fallback translations.
- d. Make design changes preferably in the header/footer to enable the user to change the locale for the website.

The strings of different components will be translated in phases preferably page wise in order in which a user is likely to visit the website which will make review and testing easier and will also help to keep track of the completed work.



Proposed flow for translations

Before moving on to the next problem, let's have a look at how accessibility should be perceived in the context of any website. Web accessibility encompasses all disabilities that affect access to the Web, including:

- auditory
- cognitive
- neurological
- physical
- speech
- Visual

Accessibility supports social inclusion for people with disabilities as well as others, such as:

- older people
- people in rural areas
- people in developing countries

Web accessibility also benefits people without disabilities, for example:

1. people using mobile phones, smartwatches, smart TVs, and other devices with small screens, different input modes, etc.

2. older people with changing abilities due to aging
3. people with “temporary disabilities” such as a broken arm or lost glasses
4. people with “situational limitations” such as in bright sunlight or in an environment where they cannot listen to audio
5. people using a slow Internet connection, or who have limited or expensive bandwidth

The second point of reducing load time addresses the first and last issue as often the internet connection available with devices such as smartwatches are either not as fast or it is quite expensive to run on them.

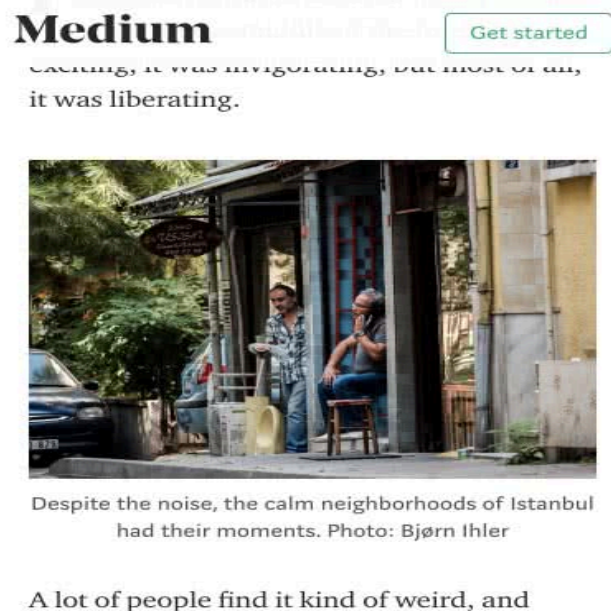
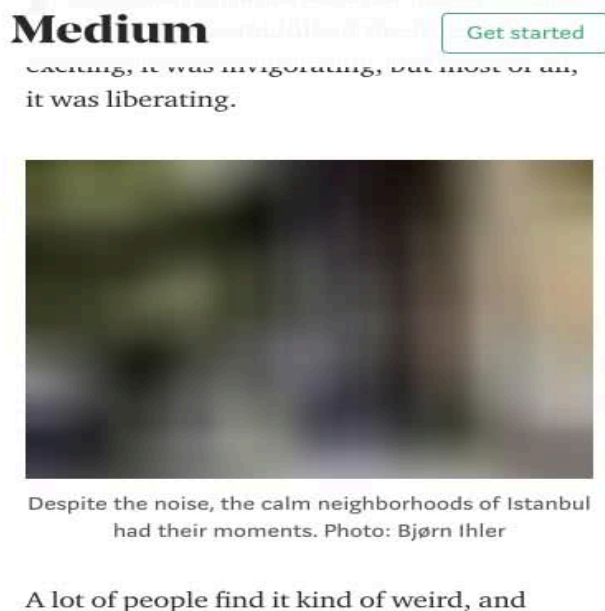
2. Optimize the CC search to reduce load time

The CC search at max does not display more than 16 images on the screen at the same time. But to display these 16 images the site fetches all the images to be shown on the page. Since these images are provided by the provider it won't be advisable to tamper with them to reduce the load times. So the best approach we can use in such a case is to lazy load the images, or if we want a better boost to the performance we can even load the components asynchronously.

To highlight the severity of the problem I had already opened an [issue](#). The page in question here ‘The Metropolitan Museum of Art’ without any filters and on the first page loads about 45MB of data which is too large just to display a handful of images. This severely affects user experience and even restricts accessibility for people with slower or expensive connections.

The possible solutions for this problem are:

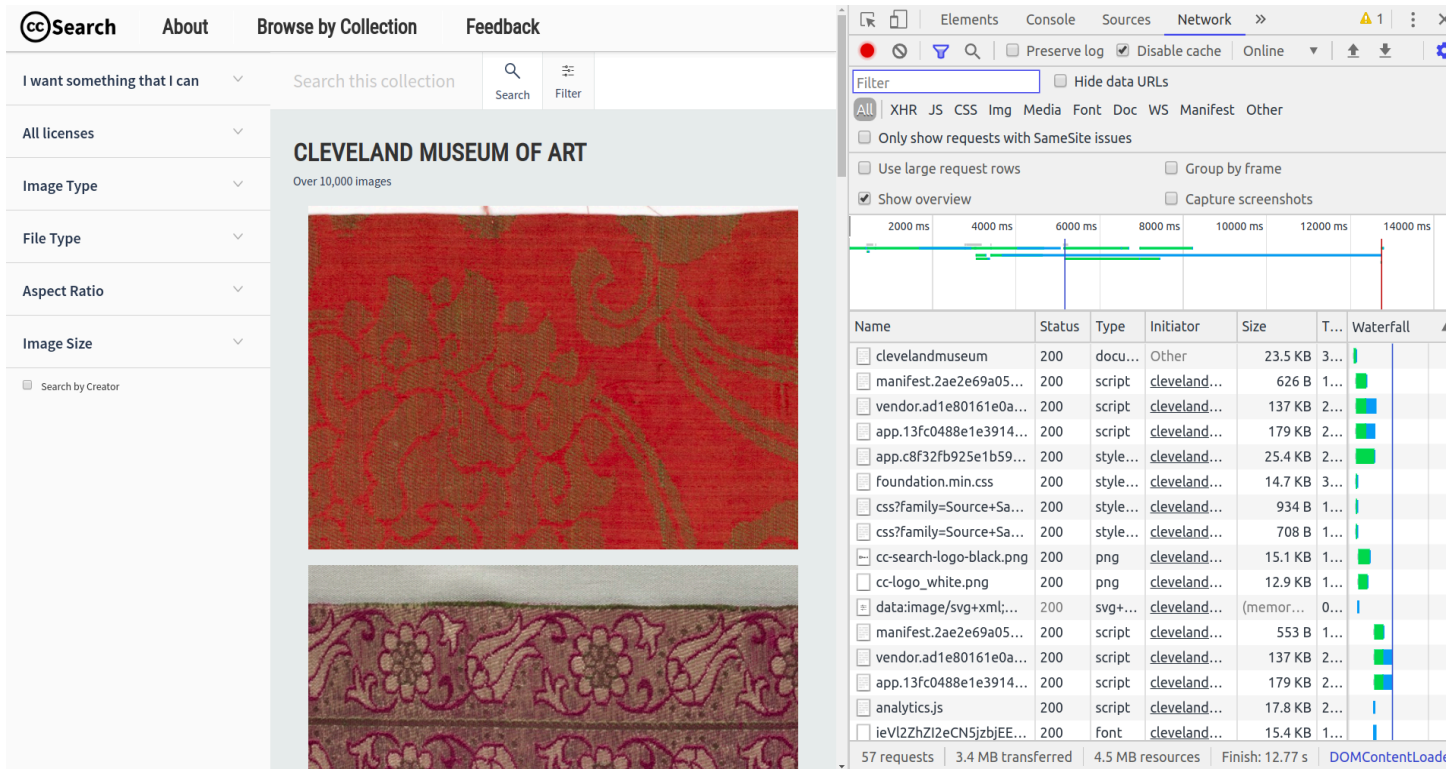
- a. Lazy load images: The best option for lazy loading of images will be the [vue-lazyload plugin](#).



An example of lazy-loaded image

The major advantage of this plugin will be a higher load speed of the website without fetching unnecessary resources over the internet. The trade-off is very minimal with only a 16KB addition to a bundle the size of 109KB. To highlight the excessive resource fetch let's look at the next screenshot of the first page of the 'Cleveland Museum of Art'. This page makes a whopping total of 57 requests at the time of

site load which is definite overkill to just display 3 images and a few static components. This greatly affects the user experience.



The first page of 'Cleveland Museum of Art' along with the network usage stats

- b. **Asynchronously load components:** To further improve load time and prevent components from loading unless necessary we can make use of asynchronous loading of components. This can be achieved by making a higher-order component wrapper for the underlying child component which asynchronously loads the component only when the need arises. Following is an example of how this can be achieved.

```
// AsyncContainer.jsx

import React, { Component } from "react"

export default function asyncContainer(importComponent) {
  class AsyncContainer extends Component {
    constructor(props) {
      super(props)

      this.state = {
        component: null
      }
    }

    async componentDidMount() {
      const { default: component } = await importComponent()

      this.setState({
        component: component
      })
    }

    render() {
      const A = this.state.component

      return A ? <A {...this.props} /> : null
    }
  }

  return AsyncContainer
}
```

```
// routes.js

...
import asyncContainer from './Containers/AsyncContainer'

const Base = asyncContainer(() => import('./Components/Base'))
const WelcomeContainer = asyncContainer(() => import('./Containers/WelcomeContainer'))

const routes = {
  component: Base,
  childRoutes: [
    {
      path: '/',
      component: WelcomeContainer,
    },
    ...
  ],
}
```

Asynchronous Container

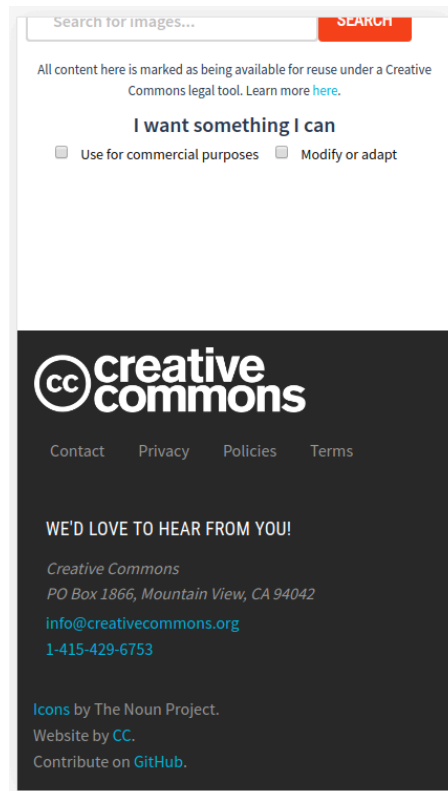
```
// Header.jsx

...
async componentDidMount() {
  const { default: RightHeader } = await import('./RightHeader')
  const { default: LeftHeader } = await import('./LeftHeader')
  this.setState({
    RightHeader: <RightHeader />,
    LeftHeader: <LeftHeader loading={false} />,
  })
}
```

An example of an asynchronously loaded component

- Enhance the aesthetics of the website keeping the mobile view in mind as well as make the site accessible to keyboard-only users

Next comes the issue of addressing the accessibility of the website on devices other than the standard working device (i.e. the PC). Let's start with the footer. The footer in the mobile view looks very inelegant and also disturbs the aesthetic look of the page. As we can discern from the image below the footer is almost as big as the page itself and the styling of the components looks distressingly out of place. Also, the use of such small-sized checkboxes makes it very difficult for the user to comfortably use the site.



The appearance of the footer in the mobile view

Another area of concern is the feedback page. Since we are using an iframe embedded google form for the feedback, which has its own separate scrolling mechanism, the effective area for scrolling on the page is very less. This area reduces even further as the device size decreases.

 A screenshot of a feedback form in mobile view. The form is contained within a light gray border. It consists of four main sections, each with a question and a response area. The first section asks: "Why do you use CC Search? (e.g. for school assignments, to find images for a blog post, etc.) *". Below this is a text input field labeled "Your answer". The second section asks: "Was CC Search useful for this purpose? *". Below this are two radio button options: "Yes" and "No". The third section asks: "How could CC Search be better and more useful to you?". Below this is another text input field labeled "Your answer". The fourth section asks: "Would you like to hear from us on important updates? If so, [click here](#)". Below this is a text input field. At the bottom of the form, there is a link that says "Report a bug". A vertical scrollbar is visible on the right side of the form, indicating that the content can be scrolled.

Feedback form in the mobile view

Another issue with the mobile view pops up in the 'About CC Search Page' which displays a table of sources. Now because of this table, the view gets a horizontal scroll which now makes the navigation more difficult as now you have added another layer of complexity regarding the navigation. This too worsens the user experience and thus cuts down on the accessibility as a result.

other licensing information we have aggregated is accurate or complete. Please independently verify the licensing status and attribution information before reusing the content. For more details, read the [CC Terms of Use](#).

Looking for the old CC Search portal? Visit <https://oldsearch.creativecommons.org>.

SOURCES

Source	Domain	#
Animal Diversity Web	https://animaldiversity.org	1
Bēhance	https://www.behance.net	6
Brooklyn Museum	https://www.brooklynmuseum.org	6
Culturally Authentic Pictorial Lexicon	http://capl.washjeff.edu	1
Cleveland Museum of Art	http://www.clevelandart.org	3
DeviantArt	https://www.deviantart.com	2

About CC Search page

Global and local menus:

TalkBack allows us to access global and local context menus, wherever we have navigated to on the device. The former provides global options relating to the device as a whole, and the latter provides options relating just to the current app/screen you are in.

To get to these menus:

- Access the global menu by quickly swiping down, and then right.
- Access the local menu by quickly swiping up, and then right.
- Swipe left and right to cycle between the different options.
- Once you've selected the option you want, double-click to choose that option.
- For details on all the options available under the global and local context menus.

The most promising way to tackle these would be simple design changes implementing which will be simple enough tasks but will greatly improve user experience on mobile devices other than the PC.

Now coming to the accessibility for keyboard-only users the major feature which every search must-have is to be able to toggle the search bar on pressing '/' key which unfortunately CC search does not provide. One of the key strengths of HTML with respect to accessibility is the built-in keyboard accessibility of

features such as buttons, form controls, and links. Generally, the tab key is used to move between controls, the Enter/Return key to select or activate controls, and occasionally other controls as needed (for example the up and down cursor to move between options in a <select> box).

However, there are components that either use non-semantic elements as buttons (or other types of control) or uses focusable controls for not quite the right purpose. In terms of making non-focusable code focusable, WAI-ARIA extends the tabindex attribute with some new values:

- `tabindex="0"` — as indicated above, this value allows elements that are not normally tabbable to become tabbable. This is the most useful value of `tabindex`.
- `tabindex="-1"` — this allows not normally tabbable elements to receive focus programmatically, e.g. via JavaScript, or as the target of links.

4. Add support for differently-abled people:

Presently the component's template does not make the best use of the HTML5 semantic controls. The only semantics that has been used as of now is the 'header', 'nav' and 'footer' sections. There aren't any complementary sections such as 'aside' or even the 'main' section.

How is the screen-reader ineffective at present in the context of cc-search?

At present, there are only 4 roles used in the cc search website 'search', 'filter', 'menubar' and 'menuitem' without that too without any `aria-label`. Also in the cases where we are dynamically updating the components and their components, we don't have any corresponding 'aria-live' attributes that will help the screen reader update the loaded VoiceOver. This makes the landmarks established by the screen reader to be not as inclusive as could have been possible had the relevant `aria`-attributes been used.

```
1 <header>
2   <h1>...</h1>
3   <nav>
4     <ul>...</ul>
5     <form>
6       <!-- search form -->
7     </form>
8   </nav>
9 </header>
10
11 <main>
12   <article>...</article>
13   <aside>...</aside>
14 </main>
15
16 <footer>...</footer>
```

Semantics code example without `aria`-attributes

Landmarks

banner
navigation
main
complementary

Corresponding VoiceOver Landmarks

The solution to this is simple and can be broken down to simple steps detailed below:

- a. Analyzing the website flow: The website flow will have to be analyzed keeping the establishment of the landmarks in mind. This will help plan the necessary components that will have to be modified or create new components if the need arises to ensure a complete experience for those with disabilities.
- b. Modification/Creation of new components: Once the components in question have been identified in the previous step, we will have to modify them to be inclusive of the relevant 'aria-attributes' or new components will have to be made along with the relevant tests and attributes to improve the site accessibility.
- c. Tests over a screen-reader and VoiceOver: The final step will involve comprehensive testing of the modified/newly created components to make sure all the features are working as desired without reducing the website's accessibility.

The major benefits of this approach are that we won't be changing too much of a live and deployed code at a time and since we already have a well-analyzed component map with us any problem in the code output we will be easily detected without causing any problems in the live website.

So a brief flow of the project will be:

1. Discuss over the design changes needed to incorporate i18n and accessibility changes.
2. Setup the i18n framework and incorporate the design changes required.
3. Translate components and make relevant accessibility changes in them.
4. Internationalize aria-labels and other accompanying attributes if required.

Note: If time permits I am also willing to abstract the manual process of checking for accessibility compliance into automated test scripts where each page will be tested for accessibility using frameworks such as JSDOM(to get the DOM elements in the page) and related tests and suitable changes can then be made along with the internationalization of the page.

Research & References

1. [Benefits of Localization/Internationalization](#)
2. [Comparison between i18n frameworks](#)
3. [Internationalization and i18n](#)
4. [Transifex](#)
5. [Integrate vue with i18n](#)
6. [Cc.i18n](#)
7. [Guide to website Internationalization](#)
8. [Asynchronous loading of components](#)
9. [Issue proposing lazy loading of images](#)
10. [MDN Accessibility](#)
11. [Mobile Accessibility](#)
12. [Multimedia Accessibility](#)
13. [WAI-ARIA Basics](#)
14. [HTML Accessibility](#)
15. [WCAG Guidelines](#)
16. [Google Accessibility](#)

Timeline & Deliverables

Timeline	Deliverables
Pre-Internship	Discuss design changes in the cc search and also those in the mobile views and make a decision on improving load times whether to go with lazy-loading, asynchronous loading or component recycling.
Week 1	Setup the i18n internationalization plugin and make the fallback English translations.
Week 2	Integrate the transifex API and develop a parsing script to parse the transifex API responses into the correct translation files. Also, add the option to change the locale in the footer (preferably).
Week 3	Map out the translations and translate the following components and pages: AboutPage (HeaderSection, FooterSection, NavSection) and HomePage (Hero-Section, HomeLicenseFilter) and SearchHelpPage.
Week 4	Figure out accessibility changes required in these pages such as the required modifications in the aria-attributes and developing new components if required to accommodate these attribute changes (also include design changes if any).
Week 5	Map out the translations and translate the following components and pages: CoollectionsPage (CollectionItem), BrowsePage (SearchGrid, SearchGridForm, SearchGridFilter), CollectionBrowsePage and PhotoDetailPage (PhotoDetails, PhotoTags, PhotoTags, RelatedImages)

Week 6	Figure out accessibility changes required in these pages such as the required modifications in the aria-attributes and developing new components if required to accommodate these attribute changes (also include design changes if any).
Week 7	Work on the lazy loading of the images, and also refactor the components to make them asynchronous to reduce even more load time if required. Also, check out the request flow to weed out any unnecessary requests made
Week 8	Map out the translations and translate the following pages: NotFoundPage and FeedBackPage
Week 9	Figure out accessibility changes required in these pages such as the required modifications in the aria-attributes and developing new components if required to accommodate these attribute changes (also include design changes if any).
Week 10	Map out the translations and translate the following components: BugReportCopyAttributionButtons, FilterChecklist, HomeLicenseFilter, ImageAttribution, ImageInfo, LegalDisclaimer, and Reuse Survey
Week 11	Figure out accessibility changes required in these pages such as the required modifications in the aria-attributes and developing new components if required to accommodate these attribute changes (also include design changes if any).
Week 12	Intensive testing of all the accessibility changes and fixing any bugs or feature breaks that are found during this time.
Week 13	Testing the translations feature and making sure each and every component is being covered and fixing any bugs.
Post-Internship	Maintaining the cc search and the features which have been worked on during this project. Help future contributors to make contributions regarding the same and work on adding new features to the cc search to provide more facilities to the users.

Questionnaire

Why are you interested in working with Creative Commons, out of all the mentoring organizations available?

Creative Commons is a remarkable organization that gives back to the community through its tireless efforts and is partnered with some of the world's most prestigious institutions. I too want to give back to the community and at the same want to experience working with those prestigious organizations. Due to this, I feel that Creative Commons is perfect for me to be working with and hence chose it over the other organizations.

Why are you interested in working on this project in particular?

While I was going on through the project list this was the first project that deeply interested me. I have already been working as full-stack web-developer for quite some time now and I feel like I can easily contribute to this code and this would project will help me become a better developer for projects with a wider audience and how to make sure that the content is made available to everyone over the web with no restrictions. I had also expressed my interest over the slack ([Slack Link](#)) and also over on one of the issues on Github ([Issue Link](#)).

Why are you well-suited to take on this project?

I have considerable experience working as a full-stack web developer and am very aware of the work I have to do to achieve the desired outcome from this project. I have a lot of experience working with the tech stack in question (Vue.js, HTML/CSS) and have researched a lot on the problem mentioned above and their respective solutions and also how different solutions weigh against each other. All this makes me quite well-suited to take this project.

Do you have any other commitments during the internship period? Provide dates, such as holidays, when you will not be available.

Since I am in my sophomore year, I don't have any commitments during the internship period and so I will be available for almost all of the internship period.

If your native language is not English, are you comfortable working closely with a mentor in English?

I am very comfortable working closely with a mentor in English as evident from the conversations I had taken part in involving the reviews of my PRs and issues.

Have you collaborated on a project remotely before?

Yes, I have collaborated on a project remotely before and have had no problem in any of the instances.

Do you plan to keep contributing to Creative Commons after the internship period is complete?

I do plan to keep contributing to Creative Commons after the internship period because I believe that since the web is a continuously evolving place, so are its needs. Thus there will always be instances where the site will need to be worked on to have more features added to address an ever-growing populace and to cater to its needs. Furthermore, this will also give me the opportunity to put my skills into practical use and give back to the community which I wholeheartedly want to keep doing through the platform of creative commons.

General Information

About Me

- Name: Ayan Choudhary
- Email: ayanchoudhary1025@gmail.com
- Phone: +91 8827254838
- Github: AyanChoudhary
- Resume: <http://bit.ly/ayanchoudhary>
- Location: Roorkee, Uttarakhand, India
- Time Zone: IST (UTC + 5:30)

I am a 20-year-old, second-year student currently enrolled in Electrical Engineering (IV Year Course) at IIT Roorkee. I developed a passion for programming and web development in my freshman year. I have been contributing to open-source regularly since about four months from now - looking over to repositories of the products I use/come across and trying to give my part of contribution back to the organization whose product has been an asset to me.

I am an active member of SDS Labs at IIT Roorkee, a bunch of passionate enthusiasts trying to foster software development culture on the campus. I have a previous experience to work as per my proposed timeline with a mentor on a winter project in SDS Labs. Also, most of our work is done according to a strict timeline with a lot of

focus on test-driven development and following best practices to structure code as per the design principles for the task.

Contact And Work Timings

- Reachable anytime through **email**, **slack** or a well-planned video session if required.
- **GPG ID:** 2AC4D174766B3A46
GPG Fingerprints: 5C6F A76B B17E F89D BFCC D343 2AC4 D174 766B 3A46
- **Timings:** UTC 5:30 AM to 8:30 AM (IST 11:00 AM to 2:00 AM)
- **Typical Working Hours:** UTC 4:30 AM to UTC 8:30 AM, UTC 10:30 AM to UTC 3:30 PM, UTC 4:30 PM to UTC 7:30 PM

Basic Information

Name & Contact Information			
Full name*	Ayan Choudhary	Email address*	ayanchoudhary1025@gmail.com
Gender/Pronouns*	Male	City, Country*	Roorkee, India
Slack username*	ayan	Phone number*	+91 8827254838
Postal address*	F-74, Govind Bhawan, IIT Roorkee, Uttarakhand, India, PIN:247667	Emergency contact information*	+91 7225914694
Websites & URLs			
GitHub*	https://github.com/AyanChoudhary	Homepage	
Twitter		LinkedIn	https://www.linkedin.com/in/ayanchoudhary/
Other code or work samples	https://github.com/sudo-rm-force/InstaVote , http://spark.iitr.ac.in/ , https://github.com/AyanChoudhary/ReProX , https://github.com/AyanChoudhary/ParElections/		
Education			
Institution	IIT Roorkee	Degree	B. Tech
Major	Electrical Engineering	Graduation Year	2022
Courses Taken	Data Structures and Algorithm, Introduction to C++, Advanced Neural Networks		

Skills

Skill name	Proficiency (1-5)	Where/how you've used this skill
Vue	5	I am also the developer of the new portfolio for the SDS Labs organization which is written in vue and Django with PostgreSQL.
Python (Django and Flask Stack)	5	The backend for Studyportal is written completely in Django with PostgreSQL and I have also used it in rewriting the met_museum script.
React	5	React is used for the frontend of both my projects of StudyPortal, InstaVote and the website for SPARK.
Java and Gradle	5	Used for my android application ParElection.
JSON	5	I have worked with many API scripts and have also written quite a few myself and most of those involved working with JSON as the format.
HTML/CSS/JavaScript	5	I have been using HTML, CSS, and JavaScript in all of my web development projects to date and have a lot of experience as a full-stack web developer.

Experience and Contributions

Brief Description	Relevant Links	Additional Notes
Rewrite MetMuseum API script with new modules and the corresponding tests	https://github.com/creativecommons/catalog/pull/278	Modified and refactored the MetMuseum API provider script to use the new modules of DelayedRequest and the ImageStore class and also to accommodate the changes in the API of MetMuseum and also wrote the corresponding tests. The reviewer was Brent Moran.
Added suggestions feature to the image response in cc catalog-api	https://github.com/creativecommons/catalog-api/pull/431	Had worked with Alden Page as the mentor to integrate the elastic search 'suggestors' feature with the API endpoint to accommodate the suggestions feature in the new cc search.
Migrate cccatalog-frontend from webpack 3 to webpack 4	https://github.com/creativecommons/catalog-frontend/pull/660	Rewrote the webpack config files along with .babelrc to migrate from webpack-3 to webpack-4. The work is still in progress. The projects are successful and the production build is successfully working and has been reviewed by Ari Madian. The development build is under progress due to an error in WebSocket which is causing a render to fail.

Wrote coverage tests for pages/client	https://github.com/reativecommons/cc-catalog-frontend/pull/627	Added coverage tests covering the whole set of pages in the pages/client directory and also modified the featureFlags file (which handles the flags variables for the components) to accommodate a fallback. The reviewers were Ari Madian and Breno Ferreira.
Wrote coverage tests for pages/server	https://github.com/reativecommons/cc-catalog-frontend/pull/635	Added coverage tests covering the whole set of pages in the pages/client directory and also modified the featureFlags file (which handles the flags variables for the components) to accommodate a fallback. The reviewers were Ari Madian and Breno Ferreira.
Implement oEmbed endpoint for the cccatalog-api	https://github.com/reativecommons/cc-catalog-api/pull/469	Added support for the oEmbed endpoint in the cccatalog-api along with providing relevant tests
Made github workflow for normalizing repos	https://github.com/reativecommons/cc-os-scripts/pull/9	Made the github workflow to automate the normalization of repos which runs every day and on every push. The reviewers were Kriti Godey and Timid Robot Zehta.
Migrate cccatalog-api test from travis CI to github workflows	https://github.com/reativecommons/cc-catalog-api/pull/448	The tests have been successfully migrated from Travis CI to GitHub workflows and is pending review from Alden Page.
Issue opened regarding improvement of cc search design and aesthetics for feedback form	https://github.com/reativecommons/cc-catalog-frontend/issues/764	This issue addressed the changes in page aesthetics and was a feature request regarding the feedback form.
Issue opened regarding improvement of load time for cc search	https://github.com/reativecommons/cc-catalog-frontend/issues/762	This issue addressed the ridiculously large load time and the unnecessary loading of resources on search.
Issue opened regarding removal of scrollbar from homepage	https://github.com/reativecommons/cc-catalog-frontend/issues/761	Minor bug report on the homepage.
Online round winning submission for MicrosoftCodefundoo++ 2019	https://github.com/sudo-rm-force/InstaVote	The project involved developing an online voting system on the blockchain. Our project was made with react as the frontend, and a hybrid server model comprising a central repository and a distributed blockchain storage.
The developer of SPARK website(official summer internship program of IIT Roorkee)	http://spark.iitr.ac.in/	I am also part of the development team of SPARK which is the official summer internship programme of IIT Roorkee.
Developer at SDS Labs	https://github.com/sdslabs	I am also a part of SDS Labs, a group of student developers that aims at fostering the culture of development and related computer science fields at IIT Roorkee.

